
BỘ TÀI LIỆU KỸ THUẬT · TÀI LIỆU 03

DAAB

Hạ tầng, Bảo mật & Vận hành



PHIÊN BẢN
1.0

NGÀY
2026-09-24

PHIÊN BẢN MÃ NGUỒN
go @ 16dc548 · python @ 6966d4b · web @
907bc8b

Mục lục

1 Mô hình triển khai

Stack Docker Compose tự host · Môi trường cloud dev trên AWS · Trạng thái triển khai

2 Build, phát hành & CI/CD

Container image · Gói phát hành · Migration cơ sở dữ liệu · Pipeline

3 Cấu hình

Các lớp cấu hình · Biến môi trường theo nhóm

4 Khả năng mở rộng

Các tầng stateless · Mô hình hàng đợi worker · Cơ sở dữ liệu & chỉ mục vector · Ràng buộc hiện tại

5 Kiến trúc bảo mật

Ranh giới tin cậy · Xác thực · Phân quyền · Mã hóa thông tin xác thực · Thiết kế truy vấn an toàn · Ghi có kiểm soát · Nhật ký audit · Tổng hợp biện pháp kiểm soát

6 Bảo vệ dữ liệu & luồng dữ liệu AI

Dữ liệu giữ lại cục bộ · Dữ liệu gửi tới mô hình bên ngoài · Cấu hình nhà cung cấp

7 Vận hành

Log & metric · Endpoint health check · Sao lưu · Tác vụ nền & phục hồi

8 Hạng mục gia cố trước khi lên production

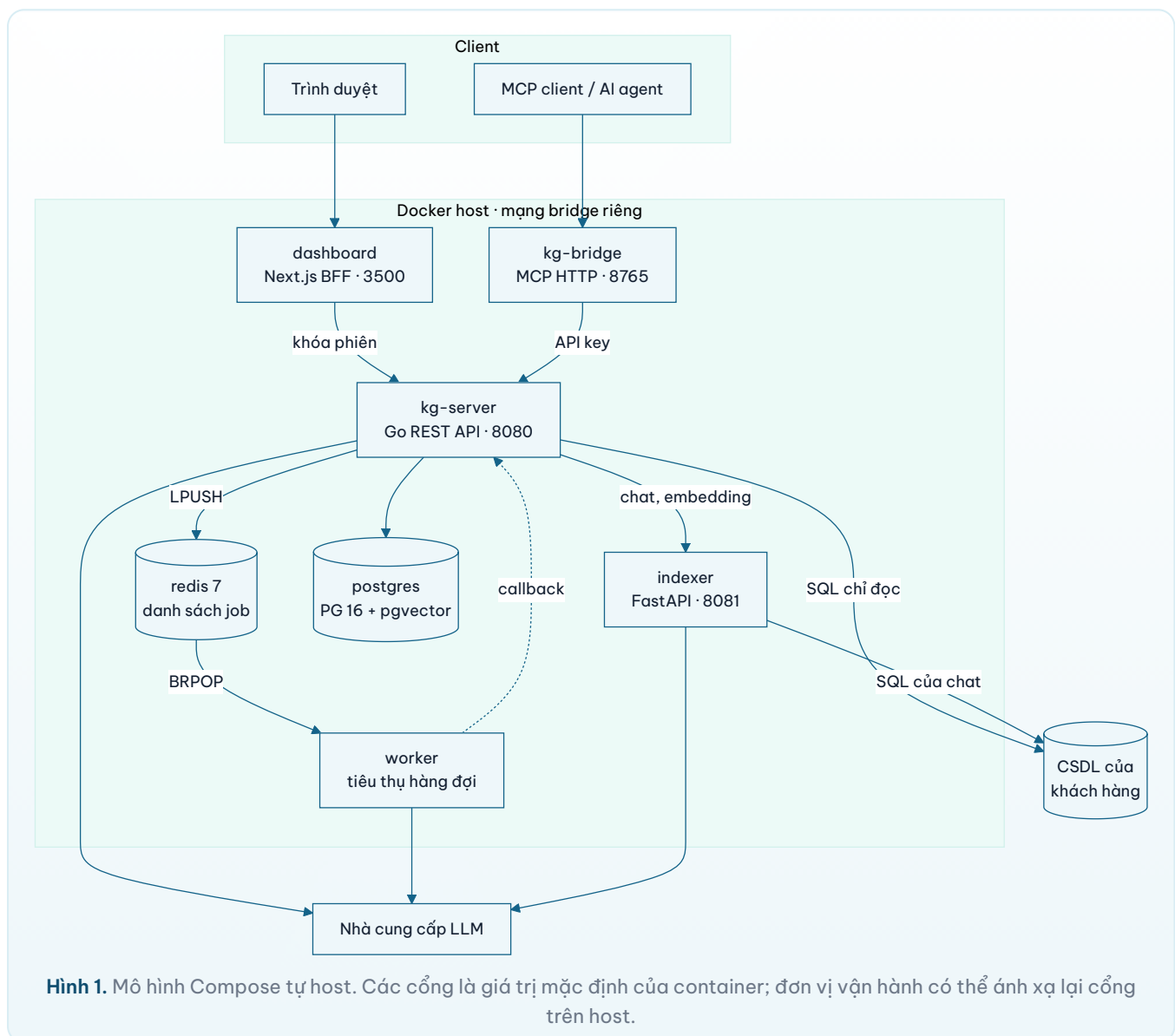
Thang trạng thái dùng trong toàn tài liệu: **LIVE** đã hiện thực, đã tích hợp và đã chạy với dữ liệu thật · **BUILT** đã hiện thực và đã tích hợp, chưa chạy với dữ liệu production · **PLANNED** nằm trong lộ trình, đang tắt bằng cờ tính năng hoặc mới ở dạng stub.

1 Mô hình triển khai

DAAB đang ở **giai đoạn tiền production (chưa triển khai production)**. Hiện nền tảng chạy dưới dạng stack Docker Compose tự host phục vụ phát triển và trình diễn, dùng một trong hai cấu hình: cấu hình phát triển build từ mã nguồn, hoặc gói phát hành với image cố định phiên bản (v1.0.0). Một môi trường cloud dev riêng trên AWS chạy nền tảng trên các dịch vụ được quản lý (ECS, RDS PostgreSQL, ElastiCache Redis, ALB); CI/CD build và deploy dịch vụ Go lên môi trường này.

1.1 Stack Docker Compose tự host

Nền tảng chạy trên một Docker host duy nhất, với một mạng bridge riêng và bảy dịch vụ: PostgreSQL 16 kèm pgvector, Redis 7, API server Go, indexer và worker Python, dashboard Next.js, và MCP bridge (dùng chung image Go nhưng khác điểm khởi chạy).



Dịch vụ	Vai trò	Trạng thái lưu trữ
postgres	Hệ thống lưu trữ gốc: graph, phiên bản, embedding, người dùng, khóa, nhật ký audit. PostgreSQL 16 + pgvector, bật data checksum.	Volume postgres_data
redis	Danh sách job (Go → Python) và trạng thái làm rõ câu hỏi trong chat có thời gian sống ngắn.	Tạm thời
kg-server	REST API (251 route), pipeline truy vấn ngôn ngữ tự nhiên, đường ghi vào CSDL nguồn, các worker nền.	Volume upload
indexer / worker	Một image Python, hai vai trò: streaming chat và dịch vụ embedding cục bộ; nạp tài liệu, OCR, trích xuất, hợp nhất thực thể, embedding bộ nhớ agent.	Volume upload, cache mô hình
dashboard	Giao diện web kèm proxy backend-for-frontend (BFF) tới API.	Stateless
kg-bridge	MCP server cho AI agent qua Streamable HTTP (image Go, điểm khởi chạy thay thế).	Stateless

Có hai cấu hình Compose. **Cấu hình phát triển** build từ mã nguồn, có hot reload cho server Go và bind-mount các repository để index mã nguồn. **Cấu hình phát hành** (`docker-compose.release.yml`) kéo ba image cố định phiên bản, không bind-mount. Cấu hình này đặt `ENVIRONMENT=selfhosted` , gần giống production: log JSON, không trả lỗi chi tiết, không có endpoint profiling, secret chỉ lấy từ biến môi trường, và tự động migration khi khởi động. Các secret cần thiết được khai báo là bắt buộc, nên stack sẽ không khởi động nếu thiếu.

1.2 Môi trường cloud dev trên AWS

Terraform trong repository (`deploy/terraform`) định nghĩa các module cho VPC, security group, RDS, ECS (kèm load balancer, image registry, autoscaling và cảnh báo), secret và một bucket cấu hình, với hai profile `staging` và `production` . Module ECS định nghĩa một dịch vụ duy nhất là API server Go. Instance Redis được quản lý (ElastiCache) cùng các task Python và dashboard của môi trường cloud dev **không** được định nghĩa trong Terraform của repository. Các thành phần này vẽ nét đứt ở Hình 2.

Mạng và tính toán (Terraform)

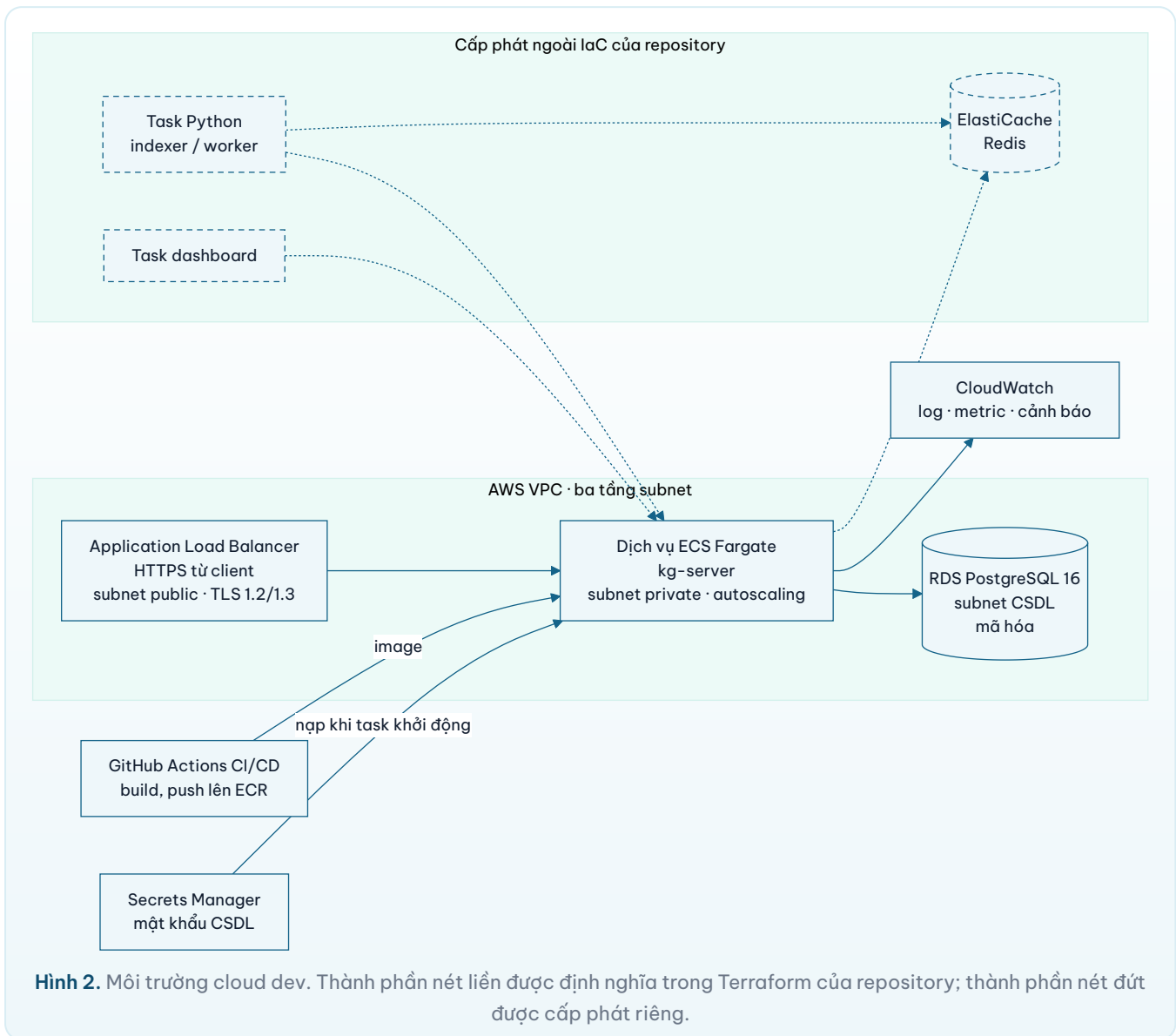
- VPC có ba tầng subnet (public, private, CSDL) và NAT gateway cho lưu lượng ra từ tầng private.
- Security group chỉ cho phép chuỗi ALB → ECS → RDS. RDS nhận lưu lượng từ security group ECS và một bastion tùy chọn.
- ALB có listener HTTPS với chính sách `ELBSecurityPolicy-TLS13-1-2-2021-06` ; HTTP chuyển hướng sang HTTPS khi có chứng chỉ. Health check gọi `/healthz` .
- ECS chạy trên Fargate, không có IP public. Deploy dùng circuit breaker, tự động rollback. Profile staging có thể dùng Fargate Spot.

Dữ liệu, secret và telemetry (Terraform)

- RDS PostgreSQL 16 có lưu trữ mã hóa và không cho truy cập public. Log truy vấn chậm, statement timeout và `pg_stat_statements` được thiết lập qua parameter group.
- Mật khẩu CSDL được sinh vào Secrets Manager và nạp vào task. SSM parameter cho tinh chỉnh runtime và cờ tính năng đã được cấp phát nhưng ứng dụng chưa đọc (provider secret cloud hiện là stub).
- Bucket cấu hình S3 có versioning, mã hóa và chặn truy cập public; workflow deploy đẩy file cấu hình lên đây nhưng ứng dụng chưa đọc cấu hình từ S3.
- Khóa mã hóa (`KG_ENCRYPTION_KEY`) không do Terraform hay workflow deploy cung cấp mà được cấp riêng; thiếu khóa, server vẫn khởi

động nhưng tắt các tính năng dùng thông tin xác thực đã mã hóa (Chương 3).

- CloudWatch nhận log group và cảnh báo về CPU, host không đạt health check, HTTP 5xx, tỷ lệ lỗi ứng dụng và độ trễ p99.



1.3 Trạng thái triển khai

Phương án triển khai	Trạng thái	Ghi chú
Compose tự host, cấu hình phát triển	LIVE	Dùng hằng ngày cho phát triển và trình diễn.
Compose tự host, gói phát hành v1.0.0	BUILT	Image cố định phiên bản, profile <code>selfhosted</code> . Smoke test bản phát hành đã hoàn tất một lần cài đặt sạch. Schema đã có thêm migration kể từ bản phát hành đó.
Môi trường cloud dev trên AWS	BUILT	Tầng dữ liệu được quản lý. CI/CD tự động deploy dịch vụ Go; việc deploy Python và dashboard chưa được tự động hóa.
Môi trường production	PLANNED	Profile Terraform <code>production</code> đã có. Chưa có khách hàng production, SLA hay số liệu quy mô đo thực tế.

2 Build, phát hành & CI/CD

2.1 Container image

Image	Cách build	Kích thước	Người dùng runtime
daab-server	Multi-stage: builder <code>golang</code> → runtime <code>alpine</code> . Binary tĩnh build với <code>CGO_ENABLED=0</code> (<code>kg-server</code> , <code>kg-bridge</code>), phiên bản và commit được gắn lúc link. Image chứa <code>config/</code> và <code>db/migrations/</code> . Một stage <code>development</code> riêng cung cấp hot reload.	≈ 38 MB	Không phải root
daab-python	Hai stage <code>python:3.12-slim</code> với <code>uv sync --frozen --no-dev</code> . Cố định wheel PyTorch chỉ-CPU và kèm Tesseract OCR có dữ liệu tiếng Việt. Một image chạy cả indexer lẫn worker; kích thước lớn là do runtime ML cục bộ (Chương 6).	≈ 1.7 GB	Không phải root
daab-dashboard	Output <code>standalone</code> của Next.js trên <code>node:20-alpine</code> . Thiết lập public của nhà cung cấp định danh được nhúng lúc build.	≈ 284 MB	Không phải root

2.2 Gói phát hành

Bản phát hành được tạo dưới dạng **gói image cố định phiên bản**: ba image được gắn tag theo commit SHA, phiên bản ngữ nghĩa và `latest`, kèm `docker-compose.release.yml`. Một script phát hành cục bộ build và gắn tag image, và có thể push lên registry riêng. File Compose nhận tiền tố registry và tag phát hành tùy chọn, nên cùng một gói có thể được cài đặt từ bất kỳ registry nào.

2.3 Migration cơ sở dữ liệu

- **86 migration SQL có đánh phiên bản**, mỗi migration là một cặp up/down do `golang-migrate` quản lý, tạo ra 54 bảng ứng dụng. Các migration bao gồm extension PostgreSQL (`uuid-oss`, `pg_trgm`, `vector`, `unaccent`), knowledge graph, kiểm soát truy cập, metadata nguồn dữ liệu, nạp dữ liệu, hợp nhất thực thể và bộ nhớ agent.
- Migration được theo dõi phiên bản và idempotent. Profile `selfhosted` và phát triển tự động migration khi khởi động, và khởi động lại không gây thay đổi.
- CLI `kg-migrate` chuyên dụng hỗ trợ `up`, `down`, `version`, `force` và `steps`. CLI này dùng cho các thay đổi do đơn vị vận hành chủ động thực hiện, chạy từ mã nguồn (Makefile) và không có trong container image.
- Migration tự động bị tắt trong các profile cloud. Bước migration trong pipeline CI chưa thực sự chạy migration (xem §8), nên thay đổi schema trên cloud hiện do đơn vị vận hành áp dụng.

2.4 Tích hợp và phân phối liên tục

Pipeline	Chức năng	Trạng thái
Go CI (<code>ci.yml</code>)	Khi push và mở pull request vào các nhánh chính: <code>golangci-lint</code> và kiểm tra <code>go mod tidy</code> ; áp dụng migration lên một container dịch vụ PostgreSQL; chạy toàn bộ bộ test với race detector và đo coverage; một test tích hợp kiểm tra cách ly giữa các dự án khi truy hồi bộ nhớ agent; build binary và Docker image.	LIVE

Pipeline	Chức năng	Trạng thái
Go deploy (workflow staging / production)	Build và push lên ECR, tạo task definition ECS mới, deploy có chờ ổn định, đẩy các file cấu hình lên S3 (ứng dụng chưa đọc, §1.2), rồi chạy health check sau deploy qua load balancer.	BUILT
Deploy cấu hình	Đẩy thay đổi cấu hình và buộc ECS deploy lại mà không cần build lại image.	BUILT
Python và dashboard	Test Python và lint/type-check dashboard chạy cục bộ; chưa có workflow CI chạy trên dịch vụ CI (hosted). Image được build bằng script phát hành. Chưa có deploy tự động lên ECS.	PLANNED

Tài sản kiểm thử tại phiên bản mã nguồn

Go: 2,753 hàm test trong 333 file test, trong đó có 16 file gắn tag integration chạy trên PostgreSQL thật. Python: 943 test được thu thập; lần chạy unit test ngày 2026-09-24 cho kết quả 922 đạt và 1 lỗi (20 test còn lại trong số đã thu thập không chạy). Dashboard: TypeScript strict, có cấu hình ESLint nhưng chưa bắt buộc (còn 27 lỗi và 28 cảnh báo); chưa có bộ test UI tự động. Test tích hợp sẽ bỏ qua khi không kết nối được CSDL, nên một lần "đạt" ở máy cục bộ tự nó không chứng minh các test này đã chạy.

3 Cấu hình

3.1 Các lớp cấu hình (dịch vụ Go)

Dịch vụ Go xác định cấu hình qua bốn lớp. Mỗi lớp ghi đè lớp trước:

- config/config.yaml** : mô hình miền. File định nghĩa schema các loại node, whitelist cạnh (59 quy tắc), quy tắc về tính đầy đủ, tìm kiếm, chiến lược và ngân sách nhà cung cấp AI, trần chi phí trích xuất, và các ngưỡng hợp nhất thực thể.
- config/environments/<env>.yaml** : mỗi profile triển khai một file (`development` , `staging` , `production` , `selfhosted`). Mỗi file đặt timeout server, connection pool, logging, nhà cung cấp hàng đợi, metric và cờ tính năng.
- Biến môi trường** có tiền tố `KG_` . Trên cloud, mật khẩu CSDL được nạp vào môi trường task từ Secrets Manager; khóa mã hóa được cung cấp riêng (§1.2).
- Bảng `system_settings`** : các giá trị ghi đè lúc runtime do quản trị viên chỉnh trong dashboard, được cache trong tiến trình.

Các dịch vụ Python dùng `pydantic-settings` có kiểu kèm giá trị mặc định; dashboard đọc một vài biến phía server cùng thiết lập public của nhà cung cấp định danh lúc build.

3.2 Biến môi trường theo nhóm

Chỉ liệt kê tên. Giá trị phụ thuộc từng lần triển khai và không bao giờ được commit. File môi trường được loại khỏi quản lý phiên bản.

Nhóm	Biến
Cơ sở dữ liệu	KG_DB_HOST KG_DB_PORT KG_DB_NAME KG_DB_USER KG_DB_PASSWORD KG_DB_SSLMODE · KG_MIGRATIONS_DIR
Hàng đợi	KG_QUEUE_PROVIDER KG_REDIS_URL KG_REDIS_QUEUE_NAME · REDIS_URL REDIS_QUEUE_NAME REDIS_KG_GENERATION_QUEUE EXTRACTION_QUEUE_NAME AGENT_CONTEXT_QUEUE_NAME

Nhóm	Biến
Secret & mật mã	<code>KG_ENCRYPTION_KEY</code> (khóa AES-256) · <code>ENCRYPTION_KEY</code> (cùng khóa, phía Python) · <code>SESSION_SECRET</code> (cookie dashboard)
Định danh dịch vụ	<code>KG_API_KEY</code> · <code>GO_API_URL</code> · <code>GO_API_KEY</code> · <code>KG_SERVER_URL</code> · <code>KG_PYTHON_URL</code>
Nhà cung cấp định danh	<code>KG_SUPABASE_URL</code> · <code>KG_SUPABASE_ADMIN_EMAILS</code> · <code>NEXT_PUBLIC_SUPABASE_URL</code> · <code>NEXT_PUBLIC_SUPABASE_ANON_KEY</code> · <code>KG_OAUTH_CLIENT_ID</code> · <code>KG_OAUTH_REDIRECT_URI</code>
MCP bridge	<code>KG_MCP_TOKEN</code> · <code>KG_MCP_TOKEN_READONLY</code> · <code>KG_MCP_AUTH_PASSTHROUGH</code> · <code>KG_MCP_REQUIRE_TLS</code> · <code>KG_PROJECT_ID</code>
AI & embedding	<code>EXTRACTION_LLM_URL</code> · <code>EXTRACTION_LLM_KEY</code> · <code>EXTRACTION_LLM_MODEL</code> · <code>ANTHROPIC_API_KEY</code> · <code>KG_EMBEDDING_API_URL</code> · <code>KG_EMBEDDING_API_KEY</code> · <code>EMBEDDING_MODEL_NAME</code> · <code>EMBEDDING_SERVICE_URL</code> · <code>KG_PLAN_CACHE</code> · <code>KG_VALUE_HINTS</code>
Tính chỉnh runtime	<code>WORKER_CONCURRENCY</code> · <code>NL_QUERY_TIMEOUT</code> · <code>STREAM_TIMEOUT</code> · <code>RESOLUTION_*</code> · <code>KG_UPLOAD_DIR</code> · <code>HF_HOME</code>
Quan sát hệ thống	<code>KG_LOG_LEVEL</code> · <code>LOG_LEVEL</code> · <code>KG_CLOUDWATCH_*</code> · <code>KG_METRICS_*</code> · <code>KG_VERBOSE_ERRORS</code>
Môi trường	<code>ENVIRONMENT</code> · <code>KG_CONFIG_DIR</code> · <code>KG_APP_CONFIG</code> · <code>NODE_ENV</code> · <code>COOKIE_SECURE</code> · <code>KG_AUTH_NOOP</code> (chỉ dùng khi phát triển)

Bật/tắt tính năng theo hướng an toàn khi lỗi

Server vẫn khởi động khi thiếu `KG_ENCRYPTION_KEY`, nhưng khi đó kết nối nguồn dữ liệu, đường ghi vào nguồn, lựa chọn nhà cung cấp AI, truy vấn ngôn ngữ tự nhiên và liên kết tài khoản đều bị tắt. Hệ thống không bao giờ lùi về lưu thông tin xác thực ở dạng không mã hóa. Cơ chế bỏ qua xác thực chỉ dành cho phát triển (`KG_AUTH_NOOP`) chỉ có hiệu lực khi môi trường là `development`.

4 Khả năng mở rộng

Các tầng phục vụ request là stateless và có thể mở rộng theo chiều ngang sau load balancer. PostgreSQL là lỗi stateful duy nhất. Công việc bất đồng bộ đi qua hàng đợi dạng list trên Redis tới một tầng worker có thể mở rộng độc lập. Chưa có số liệu thông lượng hay mức đồng thời nào được đo dưới tải production, và tài liệu này không đưa ra con số nào như vậy.

4.1 Thành phần mở rộng theo chiều ngang

- **kg-server (Go), stateless.** Toàn bộ trạng thái nằm trong PostgreSQL. Xác thực là một lần tra cứu khóa đã băm cho mỗi request, không có phiên phía server. Profile ECS trong Terraform tự động mở rộng theo CPU, bộ nhớ và số request trên mỗi target (staging 1–3 task; profile production 2–10).
- **kg-bridge, dashboard BFF và Python indexer, stateless.** Bridge dựng một MCP server riêng cho mỗi phiên và mang thông tin xác thực theo từng request. Phiên dashboard là cookie mã hóa do trình duyệt giữ. Indexer nạp mô hình embedding một lần cho mỗi tiến trình.
- **Python worker, stateless.** Nhiều worker có thể cùng tiêu thụ các list Redis. `BRPOP` giao mỗi message cho đúng một consumer.
- **PostgreSQL, stateful.** Mở rộng theo chiều dọc. Profile RDS hỗ trợ tự động mở rộng dung lượng lưu trữ và Multi-AZ. Logical replication được bật trong parameter group để dùng cho read replica hoặc change-data-capture sau này.

■ 4.2 Mô hình hàng đợi worker

Server Go đẩy các message job dạng JSON vào bốn list Redis bằng `LPUSH`, và Python worker tiêu thụ chúng bằng lệnh chặn `BRPOP` theo thứ tự vào trước ra trước. Bốn list lần lượt phục vụ: index repository mã nguồn (`enram:kg:indexing`); bản nháp tài liệu, trích xuất file tải lên và đồng bộ connector (`enram:kg:generation`); trích xuất và hợp nhất thực thể ở mức chunk (`enram:extraction`, list duy nhất có thể cấu hình số consumer song song); và embedding bộ nhớ agent (`enram:agent_context_embed`).

Các thao tác đồng bộ chạy lâu được tách khỏi đường xử lý request. Đồng bộ schema chạy trên một engine job trong tiến trình với 3 worker, 3 lần thử lại và backoff 30 s. Truy vấn ngôn ngữ tự nhiên được gửi bất đồng bộ và lấy kết quả bằng long-poll. Phản hồi chat được stream qua Server-Sent Events, tối đa 3 stream chat đồng thời cho mỗi người dùng.

■ 4.3 Cơ sở dữ liệu và chỉ mục vector

- **Lưu trữ graph** dùng một bảng node duy nhất với cột phân biệt loại và thuộc tính `JSONB`. Thuộc tính có chỉ mục GIN, tiêu đề có chỉ mục GIN trigram, và có một `tsvector` toàn văn có trọng số. Mỗi lần cập nhật đều thêm một dòng lịch sử vào bảng riêng. Duyệt graph dùng CTE đệ quy có giới hạn (độ sâu ≤ 3).

- **Tìm kiếm vector** dùng ba cột pgvector:

- embedding của node và chunk, `vector(384)` với chỉ mục HNSW cosine;
- embedding bộ nhớ agent, `vector(384)` với chỉ mục HNSW;
- embedding mô tả bảng, `vector(1536)` với chỉ mục IVFFlat.

Tìm kiếm lai chạy song song nhánh từ vựng và nhánh vector, rồi trộn kết quả (RRF) bằng Reciprocal Rank Fusion. Không có cơ sở dữ liệu vector riêng nào phải vận hành, bảo mật hay sao lưu.

- **Connection pool** được cấu hình theo từng profile (ví dụ 25 kết nối mở và 10 kết nối rảnh trong `selfhosted`). Truy vấn vào CSDL của khách hàng dùng một pool mới, một kết nối, cho mỗi truy vấn, nên CSDL khách hàng chậm không thể làm cạn pool của chính nền tảng.

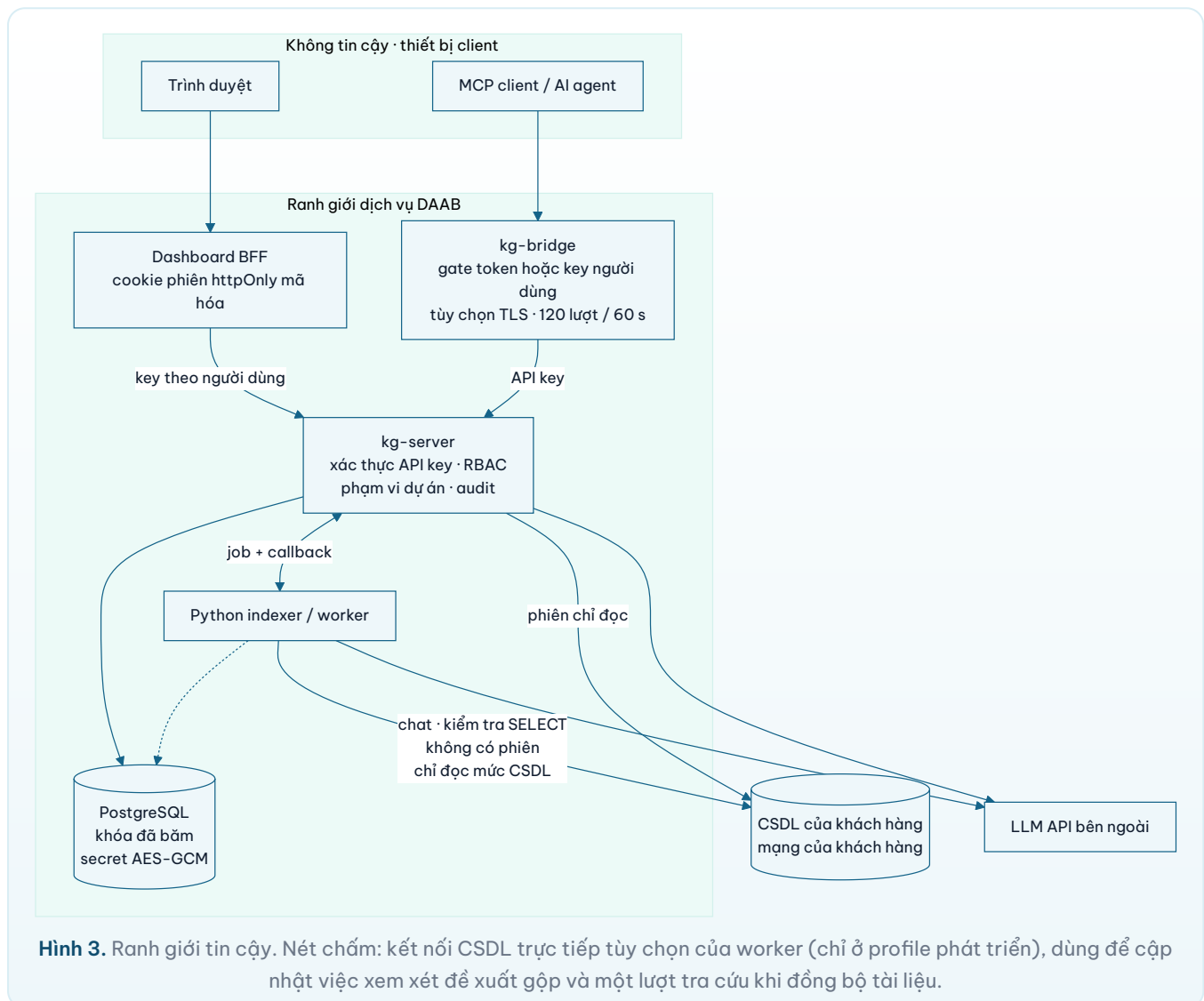
■ 4.4 Ràng buộc hiện tại (lộ trình)

Ràng buộc	Hạng mục lộ trình
Mức đồng thời của worker mang tính hợp tác: một tiến trình với nhiều consumer bất đồng bộ. Embedding cục bộ tốn CPU sẽ chặn event loop trong lúc chạy.	Chuyển embedding sang thread pool hoặc process pool, hoặc sang dịch vụ embedding của indexer. Mở rộng bằng cách thêm bản sao worker thay vì tăng mức đồng thời trong tiến trình.
Hàng đợi giao nhận theo kiểu at-most-once. <code>BRPOP</code> xóa message, và chưa có cơ chế xác nhận (acknowledgement), visibility timeout hay dead-letter queue cho job của worker. Worker bị sập giữa chừng sẽ làm mất job đó. Lỗi của từng mục trong một job được cô lập và báo cáo.	Áp dụng ngữ nghĩa hàng đợi tin cậy (ví dụ consumer group của Redis Streams hoặc SQS): xác nhận, bộ đếm thử lại, dead-letter queue, và một tác vụ quét định kỳ đẩy lại các lượt trích xuất bị treo. Hàm quét đã có nhưng chưa được lên lịch.
Giới hạn tốc độ (rate limiting) của MCP bridge nằm trong tiến trình, nên giới hạn thực tế nhân lên theo số bản sao.	Trạng thái giới hạn tốc độ dùng chung (lưu trên Redis).
Worker nền (liên kết chunk, lưu giữ bộ nhớ, giám sát heartbeat) khởi động trong mọi bản sao API.	Bầu leader hoặc dùng một task lập lịch riêng trước khi chạy nhiều bản sao API.

5 Kiến trúc bảo mật

Request tới dữ liệu nền tảng qua ba điểm vào: dashboard BFF, MCP bridge, và REST API được gọi trực tiếp bằng API key. Cả ba đều hội tụ về Go API, nơi phân quyền truy cập dữ liệu nền tảng; ngoại lệ là các route tiến độ realtime (§8, mục 7). Thông tin xác thực CSDL của khách hàng được mã hóa khi lưu trữ. Trên đường truy vấn qua API và MCP, mô hình AI không bao giờ trực tiếp thực thi SQL: mô hình tạo ra một query plan có cấu trúc, rồi mã tắt định chuyển plan đó thành SQL chỉ đọc, có tham số hóa. Chat trên dashboard được xử lý khác: Python indexer tự kết nối tới CSDL của khách hàng (§5.5).

5.1 Ranh giới tin cậy



5.2 Xác thực

- **API key** **LIVE**. Key được sinh từ nguồn ngẫu nhiên mật mã, và chỉ lưu hash SHA-256 cùng một tiền tố hiển thị ngắn; bản rõ chỉ hiển thị một lần. Mỗi key có vai trò, phạm vi dự án tùy chọn, khả năng thu hồi và theo dõi lần dùng gần nhất. Mọi route `/api/v1` đều yêu cầu key, trừ hai endpoint đăng nhập.
- **Đăng nhập bằng mật khẩu** **LIVE**. Mật khẩu lưu dưới dạng hash bcrypt, và người dùng không tồn tại đi qua một nhánh cân bằng thời gian xử lý. Tài khoản bị khóa sau 5 lần thử sai, quản trị viên có thể mở

khóa, và có thể buộc đổi mật khẩu. Mỗi lần đăng nhập bằng mật khẩu tạo một **khóa phiên theo người dùng** và thu hồi key trước đó, qua đó đảm bảo chỉ có một phiên dashboard hoạt động.

- **Phiên dashboard** **LIVE**. Cookie mã hóa `iron-session` (`httpOnly`, `SameSite=lax`, `secure` trong production) chỉ mang key của người dùng bên trong cookie đã mã hóa, nên script trên trình duyệt không bao giờ thấy API key. Khi key bị thu hồi ở phía upstream, cookie bị hủy.
- **Đăng nhập một lần (SSO)** **BUILT**. Tùy chọn. JWT của nhà cung cấp định danh được xác minh với JWKS đã cache, có kiểm tra issuer và audience. Người dùng được cấp tài khoản ngay khi đăng nhập lần đầu với vai trò viewer; quản trị viên được chỉ định qua cấu hình.
- **MCP bridge** **LIVE**. Hai chế độ:
 - *gate-token*: một token dùng chung phải dài tối thiểu 32 byte (sau khi giải mã hex hoặc base64url) hoặc 43 ký tự, được kiểm tra lúc khởi động. Một token thứ hai tùy chọn cấp phạm vi chỉ đọc.
 - *passthrough* (đa tenant): mỗi bên gọi xuất trình DAAB API key của chính mình, key này được chuyển tiếp theo từng request, nên API áp dụng đúng vai trò và dự án của bên gọi đó.

Bridge không tự kết thúc TLS. Bridge cảnh báo khi bind vào địa chỉ không phải loopback, và từ chối khởi động khi không có TLS nếu `KG_MCP_REQUIRE_TLS=true`.

- **Liên kết tài khoản AI** **BUILT**. OAuth với PKCE. Token được mã hóa khi lưu trữ và được một worker nền làm mới.

■ 5.3 Phân quyền

- **Vai trò**. Key mang một trong các vai trò `admin`, `developer`, `agent` hoặc `viewer`. Thành viên dự án mang vai trò `admin`, `developer` hoặc `viewer`, theo phân cấp cộng dồn.
- **Middleware phân giải dự án (project resolution)** xác định dự án hiệu lực cho mọi request `/api/` từ dự án mặc định của key, danh sách dự án của key hoặc tư cách thành viên của người dùng. Middleware yêu cầu quyền ghi đề tường minh và chặn mọi thao tác thay đổi trên dự án đã lưu trữ.
- **Kiểm tra ở tầng handler** xác minh lại quyền truy cập dự án đối với các định danh truyền trong body request và trên các route theo phạm vi dự án; một số route đọc theo ID (lịch sử node, dàn ý tài liệu và backlink) cũng kiểm tra dự án của tài nguyên. Thao tác ghi lên node, cạnh, nguồn dữ liệu, ghi vào nguồn và connector yêu cầu vai trò dự án tối thiểu với key gắn với người dùng.
- **Phạm vi chỉ đọc của MCP** được áp dụng cho từng tool lúc điều phối. Tool được phân loại đọc hoặc ghi, và tool ghi không mang gợi ý chỉ đọc, nên các MCP client tuân chuẩn sẽ hiện lời nhắc xác nhận trước khi gọi.
- Áp dụng thống nhất bước kiểm tra theo dự án cho mọi thao tác đọc theo ID, và cho service key và agent key không gắn với người dùng, là một hạng mục gia cố (Chương 8, mục 4).
- **Ghim dự án** trong bridge ghi đề các định danh dự án do mô hình cung cấp. Cơ chế này bảo vệ trước nhầm lẫn của mô hình nhưng không phải ranh giới tenant; ranh giới tenant được áp dụng bởi phạm vi của API key.

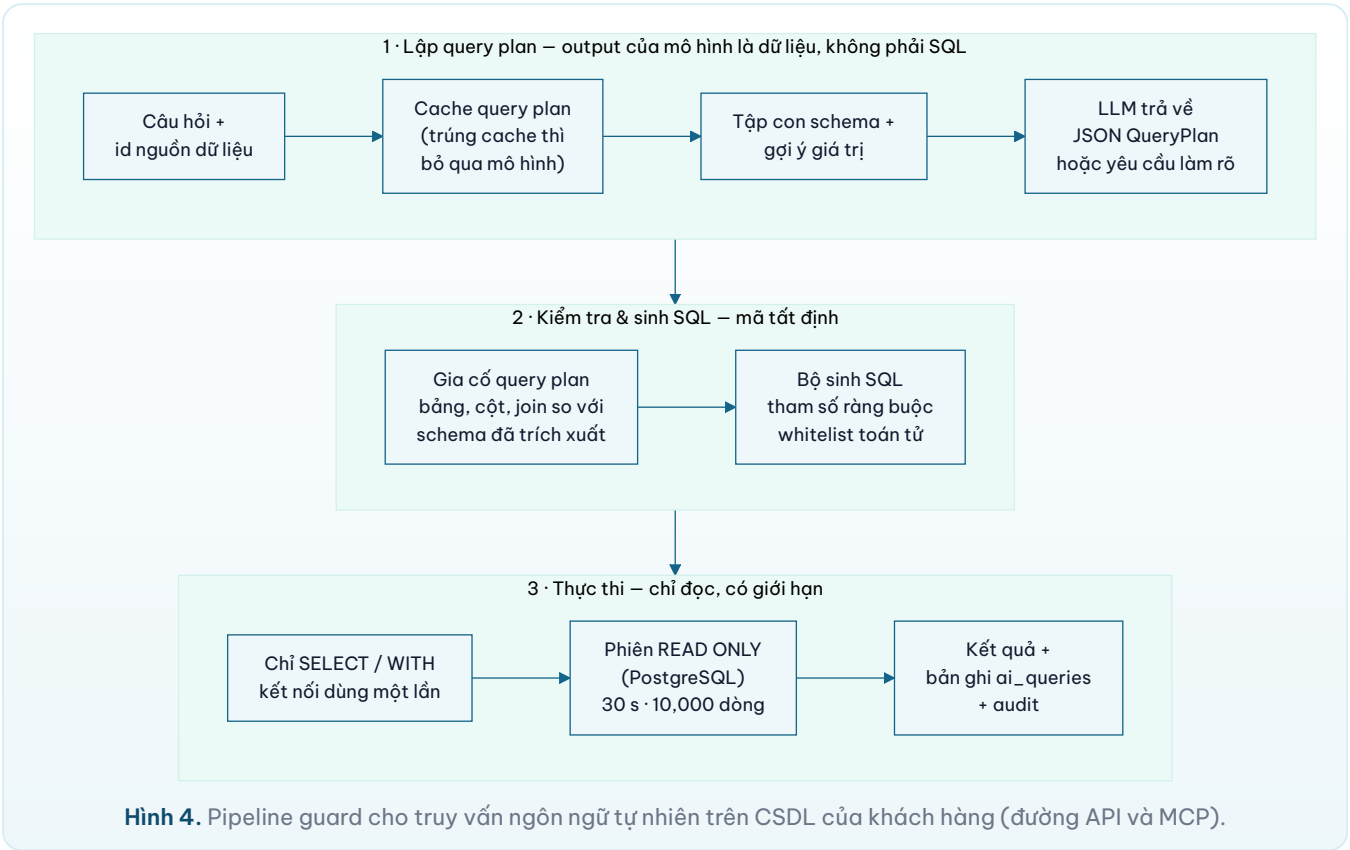
■ 5.4 Mã hóa thông tin xác thực

Chuỗi kết nối CSDL của khách hàng, API key của nhà cung cấp AI, token OAuth và thông tin xác thực connector được mã hóa bằng **AES-256-GCM**. Server dùng một khóa 32 byte cung cấp lúc runtime, sinh nonce ngẫu nhiên 12 byte mới cho mỗi lần mã hóa, và lưu `nonce || ciphertext || tag`, nên mọi hành vi sửa đổi trái phép đều bị phát hiện. Việc giải mã chỉ diễn ra trong bộ nhớ, đúng lúc mở kết nối. Chế độ chat của dashboard chuyển chuỗi kết nối *vẫn đang mã hóa* sang dịch vụ Python, vốn giữ cùng khóa. Với chat trên dashboard, server còn chuyển key của nhà cung cấp AI được chọn sang dịch vụ Python trong một

header request qua mạng nội bộ, và dịch vụ Python gọi trực tiếp nhà cung cấp. Việc thu gọn quyền nắm giữ khóa và xác thực các lượt gọi giữa dịch vụ nội bộ được liệt kê ở §8.

5.5 Thiết kế truy vấn an toàn

Pipeline truy vấn ngôn ngữ tự nhiên phía sau API và MCP tool `kg_query_datasource` được thiết kế để mô hình không bao giờ viết SQL có thể thực thi:



- Query plan, không phải SQL.** Mô hình xuất ra một `QueryPlan` có kiểu (bảng, join, bộ lọc, phép tổng hợp, nhóm, sắp xếp, giới hạn). Mọi giá trị đều trở thành tham số ràng buộc. Toán tử phải nằm trong whitelist, và số ít dạng biểu thức được phép (khoảng thời gian, truy vấn con vô hướng) phải khớp với ngữ pháp đóng.
- Neo vào schema.** Query plan được sửa và kiểm tra dựa trên schema *đã trích xuất*: sự tồn tại của cột, nâng cấp và khử trùng lặp join, tính hợp lệ của mệnh đề ON, và giới hạn 100 cho truy vấn xếp hạng. Nếu thực thi thất bại, pipeline thử lại một lần với plan đơn giản hóa.
- Guard khi thực thi.** Câu lệnh được sinh ra phải bắt đầu bằng `SELECT` hoặc `WITH`. Mỗi truy vấn dùng một pool mới chỉ có một kết nối. Với nguồn PostgreSQL, phiên còn được đặt `READ ONLY` ở mức cơ sở dữ liệu. MySQL, MariaDB và SQL Server dựa vào bộ sinh tắt định và bước kiểm tra tiền tố câu lệnh. Timeout 30 s và giới hạn cứng 10,000 dòng (kèm cờ `truncated`) áp dụng cho mọi dialect.
- Đặc quyền tối thiểu tại nguồn.** Khách hàng được khuyến nghị đăng ký thông tin xác thực CSDL chỉ đọc, là guard mạnh nhất và độc lập với DAAB.

Chat trên dashboard có thêm hai chế độ, triển khai trong Python indexer, vốn tự mở kết nối tới CSDL của khách hàng (Hình 3). Chế độ *fast* dùng cùng cách tiếp cận lập query plan rồi sinh SQL với giá trị ràng buộc. Chế độ *agentic* cho phép mô hình tự soạn câu lệnh `SELECT` trong một vòng lặp tool. Các câu lệnh này được kiểm tra bởi một bộ kiểm tra câu lệnh (một câu lệnh duy nhất, chỉ `SELECT / WITH`, danh sách chặn các từ khóa thay đổi dữ liệu, giới hạn độ dài) và kết quả bị cắt còn 50 dòng sau khi thực thi và 20 dòng cho mỗi kết quả tool. Cả hai chế độ Python đều chưa đặt phiên chỉ đọc ở mức cơ sở dữ liệu. Việc siết chặt cả hai được liệt kê ở §8.

5.6 Ghi có kiểm soát

Thao tác ghi vào CSDL của khách hàng **mặc định bị tắt** và phải qua bốn lớp kiểm soát độc lập:

- 1. **Bật theo từng nguồn.** `allow_writes` là false cho tới khi quản trị viên bật.
- 2. **Whitelist bảng.** Chỉ các bảng trong danh sách mới ghi được. Danh sách rỗng nghĩa là không ghi được bảng nào.
- 3. **Kiểm tra hợp lệ** dựa trên metadata đã trích xuất: cột bắt buộc và cột không xác định, kiểu dữ liệu và độ dài tối đa.
- 4. **Audit.** Mọi lần thử, dù thành công, bị từ chối hay thất bại, đều được ghi vào `source_write_audit`.

Thao tác ghi nhiều bước được biểu diễn thành các **playbook** có tên và phiên bản. Playbook là định nghĩa bước dạng khai báo phía server, được đề xuất một cách tắt định (không dùng mô hình) và được quản trị viên dự án phê duyệt. Playbook thực thi trong **một giao dịch CSDL duy nhất**. Sau mỗi lần đồng bộ schema, các playbook có schema đích bị thay đổi sẽ tự động bị đánh dấu *stale*. Đường ghi hiện hỗ trợ nguồn PostgreSQL.



5.7 Nhật ký audit

Các hoạt động liên quan tới bảo mật được ghi vào các bảng chuyên dụng: `audit_trail` (tạo node, thay đổi thiết lập, đồng bộ schema, gửi truy vấn); `kg_audit_log` (đọc lịch sử hội thoại); `source_write_audit` (mọi lần thử ghi vào CSDL của khách hàng); `knowledge_node_versions` (lịch sử phiên bản do trigger CSDL ghi ở mỗi lần cập nhật; thao tác sửa graph chỉ thêm phiên bản, còn khi sinh lại một tài liệu thì các mục và chunk của tài liệu đó bị thay thế cùng các dòng phiên bản của chúng); và `ai_usage_logs` cùng các bảng ngân sách (mức sử dụng mô hình theo từng request và phân bổ chi phí). Quản trị viên truy vấn nhật ký audit và lịch sử theo từng thực thể qua API.

5.8 Tổng hợp biện pháp kiểm soát

Biện pháp kiểm soát	Cách triển khai	Trạng thái
Lưu trữ khóa	API key băm SHA-256; mật khẩu bcrypt	LIVE
Mã hóa secret khi lưu trữ	AES-256-GCM cho DSN, key nhà cung cấp, thông tin xác thực OAuth và connector	LIVE
Bảo mật phiên	Cookie <code>httpOnly</code> mã hóa; một phiên dashboard hoạt động duy nhất; buộc đổi mật khẩu	LIVE

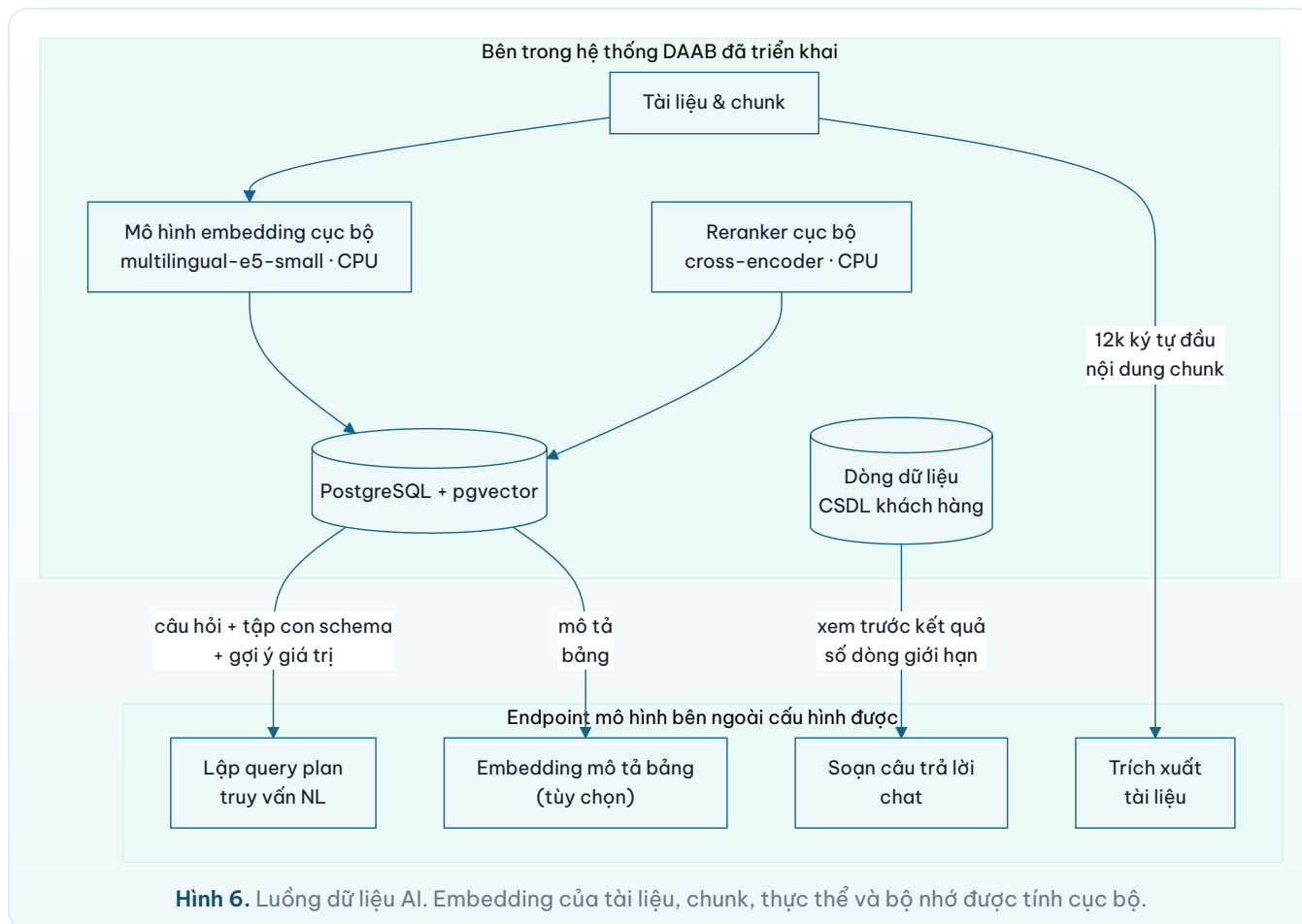
Biện pháp kiểm soát	Cách triển khai	Trạng thái
Chống dò mật khẩu	Khóa đăng nhập sau 5 lần thất bại	LIVE
Cách ly dự án	Middleware dự án, kiểm tra ở handler theo phạm vi dự án, kiểm tra lại ở mức body; một test cách ly trong CI (truy hồi bộ nhớ agent). Kiểm tra thống nhất theo ID: §8, mục 4	LIVE
Kiểm soát truy cập theo vai trò	Vai trò của key cùng vai trò thành viên theo từng dự án trên các đường ghi	LIVE
Phạm vi chỉ đọc của MCP	Token riêng; tool ghi bị từ chối lúc điều phối	BUILT
Giới hạn tốc độ MCP	120 lượt gọi mỗi 60 s cho mỗi định danh (trong tiến trình)	LIVE
Giới hạn tốc độ toàn API	Chưa áp dụng (§8)	PLANNED
An toàn truy vấn	Đường API và MCP: sinh SQL dựa trên query plan, tham số ràng buộc, phiên chỉ đọc (PostgreSQL), timeout, giới hạn số dòng	LIVE
Ghi có kiểm soát	Bật theo lựa chọn, whitelist, kiểm tra hợp lệ, audit, playbook đã phê duyệt chạy trong một giao dịch (khối lượng nhỏ)	LIVE
Bảo mật đường truyền	TLS trên ALB (cloud); tùy chọn bắt buộc TLS trên bridge	BUILT
Gia cố container	Người dùng không phải root, binary Go tĩnh, base image tối giản	LIVE
Kiểm soát thông tin lỗi	Tắt lỗi chi tiết, log body request và endpoint profiling trong các profile gần giống production	LIVE

6 Bảo vệ dữ liệu & luồng dữ liệu AI

DAAB tính embedding và rerank bằng các mô hình CPU cục bộ bên trong hệ thống triển khai, nên nội dung tài liệu không bao giờ được gửi cho bên thứ ba để tính embedding. Mô hình ngôn ngữ bên ngoài chỉ được gọi ở một số bước cụ thể, và mỗi bước chỉ thấy một tập con dữ liệu có giới hạn. Mọi nhà cung cấp đều cấu hình được, kể cả endpoint tương thích OpenAI tự host.

6.1 Dữ liệu giữ lại cục bộ

Embedding: các mục tài liệu, chunk, thực thể và bản ghi bộ nhớ agent được embedding bằng `intfloat/multilingual-e5-small` (384 chiều) trên CPU trong dịch vụ Python, và truy vấn tìm kiếm cũng được embedding bởi chính dịch vụ cục bộ này. **Rerank:** các ứng viên hợp nhất thực thể được lọc bằng một cross-encoder cục bộ. **OCR và phân tích tài liệu:** Tesseract (bao gồm tiếng Việt) và OCR bố cục cục bộ chạy trong worker, và file tạm từ các lần đồng bộ connector bị xóa sau khi xử lý. **Thực thi truy vấn:** SQL chạy từ hệ thống DAAB tới CSDL của khách hàng, kết quả chỉ được lưu trong CSDL riêng của DAAB và trả về cho bên gọi.



6.2 Dữ liệu gửi tới mô hình bên ngoài

Bước	Dữ liệu gửi tới mô hình	Điều kiện kích hoạt
Trích xuất tài liệu	12,000 ký tự đầu tiên của mỗi tài liệu mới hoặc đã thay đổi (chủ đề, thực thể, bảng được tham chiếu). Cơ chế khử trùng lặp theo hash nội dung đảm bảo một file không bao giờ bị gửi hai lần.	Mỗi lần nạp
Trích xuất thực thể ở mức chunk	Toàn bộ nội dung chunk (khoảng 1,200 ký tự mỗi chunk, tối khoảng 1,500 với mục chỉ có một chunk) để lấy thực thể và quan hệ có kiểu.	Khi được yêu cầu rõ ràng, có trần chi phí
Xác minh hợp nhất thực thể	Tên và mô tả ngắn của thực thể ứng viên, theo từng lô nhỏ.	Sau khi trích xuất
Lập query plan cho truy vấn NL	Câu hỏi, tập con liên quan của schema đã trích xuất (bảng, cột, kiểu, khóa) và một vài giá trị phân loại lấy mẫu dùng làm gợi ý chính tả (cấu hình được).	Mỗi truy vấn không được trả lời từ cache query plan
Soạn câu trả lời chat	Câu hỏi và bản xem trước kết quả (khoảng 2,000 ký tự ở chế độ fast; tối đa 20 dòng mỗi kết quả tool ở chế độ agentic) để tóm tắt, định dạng và rút ra nhận định.	Chat trên dashboard
Embedding mô tả bảng	Mô tả bảng trong schema, dùng để chọn bảng trong schema rất lớn.	Tùy chọn, chỉ khi được cấu hình

Khi một AI agent gọi DAAB qua MCP, kết quả agent nhận được sẽ do **chính mô hình của agent** xử lý, nằm ngoài ranh giới của DAAB. Dữ liệu nào được chuyển tiếp tới mô hình đó do phạm vi của key mà agent

nắm giữ quyết định.

■ 6.3 Cấu hình nhà cung cấp

- **Danh mục nhà cung cấp.** Nhà cung cấp AI được đăng ký trong CSDL với thông tin xác thực đã mã hóa. Server định tuyến mỗi request qua một bộ chọn theo thứ tự ưu tiên, có failover và circuit breaker. Có thể gán mô hình theo từng chức năng, như lập query plan, trích xuất hoặc xác minh.
- **Giao diện được hỗ trợ:** Anthropic API, gói Claude subscription đã liên kết (OAuth), và bất kỳ endpoint **tương thích OpenAI** nào. Lựa chọn thứ hai cho phép khách hàng trả DAAB tới một nhà cung cấp trong khu vực hoặc một server mô hình tự host, để không có dữ liệu nào rời khỏi môi trường của họ.
- **Kiểm soát chi phí.** Các lượt trích xuất được ước tính trước khi bắt đầu và bị từ chối nếu vượt trần theo lượt và theo tài liệu. Mỗi request đi qua tầng AI của server đều mang loại request và định danh lượt chạy để phân bổ chi phí. Hai đường gọi nhà cung cấp trực tiếp, không được ghi vào nhật ký sử dụng đó và không có circuit breaker của tầng này: endpoint trích xuất trực tiếp tùy chọn (`EXTRACTION_LLM_URL`) dùng cho trích xuất ở mức chunk, xác minh và phân xử, và chat trên dashboard trong dịch vụ Python. Cache query plan và nhánh xử lý nhanh cho kết quả rỗng giúp tránh các lượt gọi không cần thiết.
- **Điều khoản xử lý dữ liệu** với từng nhà cung cấp bên ngoài phụ thuộc vào lựa chọn triển khai của khách hàng: **TO BE COMPLETED**

7 Vận hành

■ 7.1 Log và metric

- **Log có cấu trúc.** Dịch vụ Go dùng `log/slog` (JSON trong các profile gần giống production) với middleware request-ID, access log và cơ chế phục hồi khi panic. Body request không được ghi log trong các profile này.
- **CloudWatch (tùy chọn).** Log có thể được đẩy theo lô lên CloudWatch Logs với thời gian lưu giữ cấu hình được, và một bộ thu thập metric trong tiến trình có thể đẩy lên CloudWatch Metrics mỗi 60 s. Profile tự host chỉ ghi ra standard output, và ECS thu output của container vào log group tương ứng.
- **Cảnh báo (Terraform).** CPU, target không đạt health check, HTTP 5xx, tỷ lệ lỗi ứng dụng và độ trễ p99.
- **Dịch vụ Python** ghi log qua thư viện chuẩn với mức log cấu hình được. Lỗi của từng mục trong pipeline nạp dữ liệu được ghi ở mức warning trở lên, không bao giờ bị bỏ qua âm thầm.

■ 7.2 Endpoint health check

kg-server, kg-bridge và Python indexer đều cung cấp `GET /healthz` (liveness) và `GET /readyz` (readiness), không yêu cầu xác thực. Readiness kiểm tra các phụ thuộc mà từng dịch vụ cần: ping CSDL với kg-server (trả 503 khi không sẵn sàng), API với bridge, và API cùng Redis với indexer. Indexer báo phụ thuộc lỗi là `degraded` trong body nhưng vẫn trả HTTP 200, nên hệ thống điều phối chưa thể dựa vào đó để chặn lưu lượng. PostgreSQL và Redis dùng health check của Compose (`pg_isready` , `redis-cli ping`), và các dịch vụ ứng dụng chờ tới khi hai dịch vụ này đạt health check. Target group của ALB dùng `/healthz` .

7.3 Sao lưu

Môi trường	Hiện trạng	Trạng thái
Cloud (RDS, profile Terraform)	Sao lưu RDS tự động trong khung giờ xác định (lưu 3 ngày ở profile staging, 14 ngày ở profile production), lưu trữ mã hóa, lan truyền tag sang snapshot, snapshot cuối và bảo vệ chống xóa trong production, và Multi-AZ trong production.	BUILT
Cấu hình (S3)	Bucket cấu hình có versioning, mã hóa, với chính sách vòng đời lưu giữ các phiên bản trước.	BUILT
Compose tự host	Dữ liệu nằm trong các Docker volume có tên (CSDL và upload). Gói phát hành chưa có quy trình sao lưu định kỳ, lưu giữ hay khôi phục.	TO BE COMPLETED
Kiểm thử khôi phục / mục tiêu RPO & RTO	Chưa được xác định.	TO BE COMPLETED

Chunk, embedding và metadata schema có thể được dựng lại từ file upload và CSDL của khách hàng; trạng thái không thể thay thế là cơ sở dữ liệu PostgreSQL và volume upload.

7.4 Tác vụ nền và phục hồi

Tác vụ	Mục đích	Trạng thái
Giám sát heartbeat job đồng bộ	Kiểm tra mỗi 60 s và đánh dấu thất bại các job đồng bộ schema không có tiến triển trong 30 phút, để giao diện không bao giờ hiển thị một job chạy mãi.	LIVE
Worker liên kết chunk	Mỗi 5 phút, tạo các cạnh tương đồng liên tài liệu cho nội dung mới nạp.	LIVE
Worker lưu giữ bộ nhớ	Khử trùng lặp và giới hạn mức tăng trưởng của bộ nhớ agent, đồng thời ghi lại thời điểm truy hồi.	BUILT
Làm mới token OAuth	Làm mới token của các tài khoản AI đã liên kết trước khi hết hạn.	BUILT
Quét trích xuất bị treo	Đẩy lại các lượt trích xuất bị kẹt giữa chừng. Hàm đã được triển khai nhưng chưa được lên lịch.	PLANNED
Tắt an toàn (graceful shutdown)	API và worker xử lý SIGINT/SIGTERM. Worker xóa các file tạm mở côi khi khởi động.	LIVE

8 Hạng mục gia cố trước khi lên production

Các hạng mục sau được lên lịch trước bất kỳ lần triển khai production nào. Không hạng mục nào đòi hỏi thay đổi kiến trúc: mỗi hạng mục gia cố một biện pháp kiểm soát sẵn có hoặc hoàn thiện phần tích hợp sẵn có.

#	Hạng mục	Phạm vi
1	Quản lý secret qua kho secret trên cloud	Hiện thực provider secret cloud trong ứng dụng (hiện là stub) để đọc được các SSM parameter và cấu hình S3 đã cấp phát, cung cấp khóa mã hóa qua infrastructure-as-code, và chuyển khóa sang dịch vụ quản lý khóa, thu hẹp quyền nắm giữ khóa (hiện dùng chung giữa dịch vụ Go và Python) về số dịch vụ ít nhất.

#	Hạng mục	Phạm vi
2	Mặc định bật TLS trên MCP bridge	Đặt <code>KG_MCP_REQUIRE_TLS</code> làm mặc định cho các bind không phải loopback, và cung cấp cấu hình reverse proxy TLS tham chiếu.
3	Giới hạn tốc độ toàn API	Áp dụng giới hạn theo key và theo IP trên REST API và endpoint đăng nhập (hiện chỉ MCP bridge và endpoint nạp dữ liệu công khai giới hạn lượt gọi, theo định danh và trong tiến trình), với trạng thái dùng chung giữa các bản sao.
4	Giới hạn phạm vi service key và phân quyền theo dự án	Áp dụng thống nhất kiểm tra vai trò và quyền sở hữu theo dự án, kể cả với agent key và service key, trên mọi thao tác theo phạm vi dự án và nguồn dữ liệu và mọi thao tác đọc theo ID.
5	Bắt buộc phiên đăng nhập trên mọi route API của dashboard	Áp dụng middleware phiên cho mọi route API của dashboard.
6	Xác thực giữa các dịch vụ	Kiểm tra thông tin xác thực trên các endpoint Python nội bộ (chat, embedding, quản trị) thay vì dựa vào vị trí mạng.
7	Kiểm tra origin cho WebSocket và realtime	Giới hạn origin WebSocket trong danh sách host đã cấu hình, và yêu cầu xác thực trên các route tiến độ realtime.
8	Loại bỏ thông tin xác thực seed dùng cho phát triển khỏi bản phát hành	Tách dữ liệu seed phát triển khỏi chuỗi migration chuẩn, và yêu cầu thông tin xác thực do đơn vị vận hành tạo lúc cài đặt.
9	Mức phơi bày mạng của dịch vụ nội bộ	Ngừng công bố cổng CSDL, Redis và indexer trong gói phát hành và bật xác thực Redis; chỉ công bố dashboard và bridge, phía sau TLS.
10	Xác nhận và thử lại job	Ngữ nghĩa hàng đợi tin cậy cho job của worker (xác nhận, thử lại, dead-letter), và lên lịch tác vụ quét trích xuất bị treo (§4.4).
11	Gia cố kiểm tra SQL	Thực thi chỉ đọc ở mức CSDL cho mọi loại CSDL nguồn được hỗ trợ (hiện chỉ PostgreSQL, trên đường API); kiểm tra dựa trên parser, mở rộng kiểm tra định danh, phiên chỉ đọc và statement timeout trên các đường chat của dashboard (Python); hướng dẫn cấp thông tin xác thực theo đặc quyền tối thiểu.
12	Bước migration trên cloud và phân phối toàn stack	Dùng binary <code>kg-migrate</code> trong các workflow deploy, bổ sung CI và phân phối lên ECS cho dịch vụ Python và dashboard, và đưa Redis được quản lý vào infrastructure-as-code.
13	Sẵn sàng chạy nhiều bản sao	Điều phối worker nền giữa các bản sao API, căn chỉnh HTTP timeout cho các route long-poll và streaming, cho readiness của indexer báo lỗi khi phụ thuộc gặp sự cố, và xác định quy trình sao lưu, khôi phục và RPO/RTO cho bản tự host.
14	Security header và đánh giá độc lập	Thêm content security policy và các security header chuẩn trên dashboard, sau đó thuê kiểm thử xâm nhập độc lập trước tenant production đầu tiên: TO BE COMPLETED
15	Hạch toán thống nhất các lượt gọi mô hình	Đưa endpoint trích xuất trực tiếp và các lượt gọi mô hình của chat trên dashboard qua cùng nhật ký sử dụng, ngân sách và circuit breaker của tầng AI trên server.

Tham chiếu mã nguồn (đường dẫn tương đối trong repository, tại phiên bản mã nguồn): `ennam.kg.go/cmd/kg-server/main.go`, `internal/middleware/`, `internal/handler/project_role.go`, `internal/crypto/aes.go`, `internal/service/source_executor.go`, `internal/bridge/`, `deploy/terraform/`, `.github/workflows/`, `docker-compose.release.yml`.